

다중 디바이스 기반 파일 동기화 프로토콜 확장 (push/pull)

- 다중 로그인 사용자의 디바이스간 파일 동기화 프로토콜 추가 (동기화 메타 데이터 없는 케이스)
 - 사용자의 첫 번째 디바이스(클라이언트)가 최초 동기화 기능 시작 (서버에 이 사용자의 동기화 기록 없는 상태)
 - 원래대로 push 기반으로 서버와 동기화
 - 동기화 완료 후 다른 디바이스에 대한 처리
 - 서버는 이 사용자의 다른 (로그인된) 디바이스로 변경 사실 알람
 - 알람받은 클라이언트는 start pull 동기화
 - 또는 서버가 바로 다른 디바이스로 push 동기화 시작
 - 사용자의 아무 디바이스(클라이언트)에서 파일 동기화가 켜진 상태이고 파일 변경 감지
 - 위의 경우와 동일 (사용자의 최초 동기화 케이스)
 - 사용자의 아무 디바이스(클라이언트)가 파일 동기화 기능 시작 (또는 동기화 켜진 상태로 로그인) (서버에 이 사용자의 동기화 기록이 남아 있는 상태)
 - 가장 복잡한 양방향 동기화 필요 (삭제 제외)
 - 빈 동기화 디렉토리로 시작한다는 가정하에 pull 기반 진행?
- 다중 로그인 사용자의 디바이스간 파일 동기화 프로토콜 개선
 - 주요 개선점
 - 동기화 메타 정보 이용 (기존은 별도 정보 없음)
 - 클라이언트는 서버와 양방향 동기화 (기존은 push 기반)
 - 클라이언트가 동기화 시작하거나 동기화가 켜진 채 로그인 한 경우의 동기화 과정
 - 서버로 cursor 정보 전송 (없으면 새로 만들고 0으로 설정)
 - 클라이언트가 시작하는 pull 동기화 세션의 시작 → 동기화 세션 상태를 pull로 설정
 - 서버
 - 이 클라이언트의 cursor가 없으면 (새로운 사용자) 리턴코드 0으로 응답 이벤트 전송
 - 클라이언트는 신규로 전체 파일 목록 기반 동기화 (push) 진행
 - 이 클라이언트의 cursor와 받은 cursor 비교
 - 두 값이 같으면 현재 클라이언트의 상태가 모두 동기화된 것으로 응답 이벤트 전송
 - 서버의 cursor값이 더 작으면 문제 발생으로 -1 리턴 코드로 응답 이벤트 전송
 - 클라이언트는 cursor 전체 파일 목록 기반 동기화 (push) 진행
 - 또는, server의 cursor보다 큰 파일 목록을 push로 동기화 진행
 - 받은 cursor에서 서버의 cursor값까지의 changelog 파일 엔트리 정보를 리스트로 생성
 - 각 changelog의 path를 key로, 전체 line을 value로 하는 HashMap entryMap<String, String> 생성
 - id 순서대로 (path, line)을 entryMap에 put (path에 대해 최신 log만 남게됨)
 - 완성된 entryMap()에서 클라이언트로 전송할 CMFileSyncEntry 타입 리스트 구성
 - 동기화 sender/receiver 간 전송하는 기존 파일 엔트리 정보에 op 추가
 - 응답 이벤트로 pull 동기화 필요 사실 알람. 이후 위의 파일 엔트리 리스트를 별도 이벤트로 전송

- serverEntryList를 stateKey를 key로 하고 Map<path, CMFileSyncPendingState>를 value로 하는 m_pullStateTable에 put → pull op가 완료될때마다 stateKey로 pendingPullMap을 찾고 여기서 해당 path의 completed 값을 true로 변경하다가 모두 true이면, 서버는 이 pendingPullMap의 pull 동기화 세션이 완료되었다는 사실을 알게됨
 - Map<CMFileSyncStateKey, Map<Path, CMFileSyncPendingState>> m_pullStateTable;
 - Map<Path, CMFileSyncPendingState> pendingPullMap
 - CMFileSyncPendingState 정의 아래 push 동기화 참조
 - 리스트가 긴 경우에는 여러 이벤트로 나누어서 반복 전송
- 클라이언트
- 서버로부터 응답 이벤트 수신하고 메시지 출력
 - 현재 최신 동기화 상태이면 여기서 종료
 - 동기화 세션 상태를 pull에서 해제
 - cursor값 문제가 있다고 하면 에러 메시지 출력
 - 추가 pull 동기화가 필요하다고 하면 디버그 메시지 출력
 - 서버로부터 0 리턴 코드 (신규 클라이언트) 받으면,
 - 기존 전체 파일 목록 스캔한 push 동기화만 진행
 - 서버로부터 -1 리턴 코드 (서버 cursor 값이 더 작음) 받으면,
 - cursor 복구를 위해, 기존 전체 파일 목록 기반 push 동기화 진행 (?)
 - 위의 경우가 아닐 때, 서버로부터 pull 동기화를 위한 파일 엔트리 리스트(serverEntryList) 수신
 - 충돌 상황을 판단하기 위해 클라이언트도 index 파일 유지 필요
 - 최소한으로 path별 마지막 동기화된 수정 시간 정보
 - <https://chatgpt.com/share/6918006b-b5ac-800f-8de0-2aa08a60b8da>
 - 자신의 동기화 디렉토리 내 파일 엔트리 리스트 생성 (clientEntryList)
 - 서버 엔트리와 클라이언트 엔트리 비교하며 op 별로 pull용 동기화 맵 객체 생성 및 초기화
 - CMFileSyncPendingState 정의
 - 아래 push 동기화 참조
 - pull용 삭제 파일 엔트리 상태 맵
 - Map<Path, CMFileSyncPendingState> m_pullDeleteMap;
 - pull용 신규 파일 엔트리 상태 맵
 - Map<Path, CMFileSyncPendingState> m_pullCreateMap;
 - pull용 업데이트 파일 엔트리 상태 맵
 - Map<Path, CMFileSyncPendingState> m_pullModifyMap;
 - 충돌이 발생하여 서버로 push 동기화 대상 엔트리 상태 맵 (클라는 op hint 추가, 결정은 서버에서)
 - Map<Path, CMFileSyncPendingState> m_pendingPushMap;
 - serverEntryList의 각 원소 serverEntry에 대해 다음 수행
 - 비교 대상 clientEntry에 대해 다음 값 구하기
 - long baseMtime = synInfo.getLastSyncedMtime(relPath): client path의 마지막 동기화 때 수정 시간 (값 없으면 -1)
 - long curMtime = synInfo.currentMtimeSecOrMinusOne(absPath): client path의 현재 수정 시간 (path 없으면 -1)
 - long serverMtime = serverEntry의 수정 시간 값

- op가 MODIFY
 - 서버가 수정했는데 클라이언트도 수정했으면 충돌
 - if(curMtime ≠ baseMtime)
 - 충돌로 clientEntry의 파일명에 "(conflict-yyyyMMdd-HH:mm:ss)" 추가
 - 변경 파일명이 이미 있으면 (conflict-yyyyMMdd-HH:mm:ss)(2)로 numbering
 - serverEntry는 m_pullCreateMap에 추가하여 신규 대상으로 처리
 - 충돌 파일 renamed clientEntry를 m_pendingPushMap에 넣어 추후 push 동기화 대상으로 함
 - else
 - serverEntry를 m_pullModifyMap에 추가
- op가 CREATE
 - 서버가 새로 만들었는데 클라이언트도 파일이 있으면 충돌
 - serverEntry를 m_pullCreateMap에 추가
 - if(curMtime ≠ -1)
 - 충돌로 clientEntry의 파일명에 "(conflict-yyyyMMdd-HH:mm:ss)" 추가
 - 변경 파일명이 이미 있으면 (conflict-yyyyMMdd-HH:mm:ss)(2)로 numbering
 - 충돌 파일 renamed clientEntry를 m_pendingPushMap에 넣어 추후 push 동기화 대상으로 함
- op가 DELETE
 - 파일이 있고 base와 다르거나, 파일이 있고 base와 같은데 서버 mtime과 다르면 충돌
 - if(curMtime == -1)
 - 이미 삭제된 상태이므로 별도 동작 안함
 - else
 - if(baseMtime == serverMtime && curMtime == serverMtime)
 - 마지막 상태 파일이 존재하므로, 파일 삭제 대상으로 clientEntry (또는 serverEntry)를 m_pullDeleteMap에 추가
 - else
 - 충돌이므로 clientEntry 파일명에 "(conflict-yyyyMMdd-HH:mm:ss)" 추가
 - 변경 파일명이 이미 있으면 (conflict-yyyyMMdd-HH:mm:ss)(2)로 numbering
 - 충돌 파일 renamed clientEntry를 m_pendingPushMap에 넣어 추후 push 동기화 대상으로 함
- 처음 만든 clientEntryList에서 m_pullUpdateMap 대상을 제외하고, 남은 path 중에 baseMtime보다 큰 curMtime인 path를 모아서 m_pendingPushMap에 추가
 - pull 동기화가 완료된다음, 클라이언트는 이 리스트를 서버로 보내서 push 동기화 진행 예정
 - 서버는 없는 엔트리는 신규로, 있는 엔트리는 수정 여부 판단해서 진행 (op hint에 삭제는 없음)
- 클라이언트는 위에서 구한 op별 entry list로 pull 동기화 진행 방안
 - m_pullDeleteMap 처리
 - 클라이언트
 - 해당 파일 삭제하고 CMFileSyncPendingState의 completed를 true로 변경
 - client-index의 해당 path 인메모리 메타 정보 삭제
 - 모든 파일 삭제 완료하고, 기존 COMPLETE_DELETE_FILES 이벤트로 리스트를 서버로 전송
 - 서버

- CMFileSyncStateKey로 m_pullStateTable에서 pendingPullMap을 찾고, pendingPullMap에서 삭제 완료된 CMFileSyncPendingState의 completed 값을 true로 변경
- pendingPullMap의 모든 value의 completed가 true가 되면,
 - COMPLETE_PULL_FILE_SYNC (신규) 이벤트를 클라이언트로 전송
 - pull 상태 관련 map들 초기화
 - pull은 서버에서 메타 정보 수정 필요 없음
- 클라이언트
 - pull 동기화 완료 이벤트 수신 처리는 아래 참조
- m_pullCreateMap 처리
 - 클라이언트
 - 서버에게 map내 path에 대해 파일 전송 요청 (requestFiles)
 - 서버
 - 요청받은 파일 클라이언트로 전송 (CM 파일 전송 서비스 이용)
 - 클라이언트
 - 파일 전송 완료 이벤트 (END_FILE_TRANSRFER, END_FILE_TRANSFER_CHAN) 서버로부터 받으면,
 - m_pullCreateMap의 해당 path의 pendingState 내 completed 값을 true로 변경
 - client-index의 해당 path 인메모리 메타 정보 업데이트 (파일 저장은 동기화 세션 완료후 예정)
 - 하나의 신규 파일 동기화 완료 의미로 COMPLETE_NEW_FILE 이벤트를 서버로 전송
 - 서버
 - COMPLETE_NEW_FILE 이벤트 수신
 - m_pullStateTable에서 pendingPullMap을 열고 pendingPullMap에서 해당 path의 pendingState의 completed 값을 true로 변경
 - pendingPullMap의 모든 pendingState에 대해 completed 값이 true가 되면,
 - COMPLETE_PULL_FILE_SYNC 이벤트를 클라이언트로 전송
 - 클라이언트
 - pull 동기화 완료 이벤트 수신 처리는 아래 참조
- m_pullModifyMap 처리
 - 클라이언트
 - CMFileSyncPullGenerator (신규) 객체 만들어 map 넘겨주기
 - https://chatgpt.com/s/cd_6921a40e7e08819192b948fba71d138b
 - 이 generator는 pull update용으로써 클라가 각 path 엔트리별 블록 체크섬 리스트를 만들어 서버로 전송하는 별도 스레드로 동작
 - 서버
 - Path 엔트리별로 basis 체크섬 리스트를 다 받으면 (END_FILE_BLOCK_CHECKSUM), 로컬 파일과 비교 하여 delta 정보와 매칭 블록 정보를 클라이언트로 전송 (기존 코드 활용) (UPDATE_EXISTING_FILE)
 - 엔트리의 모든 delta와 매칭블록을 전송한 후, END_FILE_BLOCK_CHECKSUM_ACK 이벤트 전송하여 완료 알림
 - 클라이언트
 - 서버가 보내는 UPDATE_EXISTING_FILE 이벤트 처리하여 기존 파일 업데이트
 - END_FILE_BLOCK_CHECKSUM_ACK을 수신하면, 파일의 업데이트 완료 처리
 - m_pullModifyMap의 해당 pendingState의 completed 값 true로 변경

- 업데이트 완료된 파일에 대해 client-index 인메모리 메타 정보 업데이트 (파일을 저장은 나중에)
 - 하나의 파일 업데이트 완료되었다는 COMPLETE_UPDATE_FILE 이벤트 서버로 전송
- 서버
 - COMPLETE_UPDATE_FILE 수신
 - pullStateTable에서 stateKey로 pendingPullMap을 찾고, pendingPullMap에서 해당 pendingState의 completed 값을 true로 변경
 - pendingPullMap의 모든 pendingState의 completed값이 true가 되면,
 - COMPLETE_PULL_FILE_SYNC 이벤트를 클라이언트로 전송
 - pendingPullMap을 clear하고 pendingPullMap을 pullStateTable에서 삭제
- 클라이언트
 - pull 동기화 완료 이벤트 수신 처리는 아래 참조
- 클라이언트가 pull 동기화 완료 (COMPLETE_PULL_FILE_SYNC) 이벤트 수신 처리
 - 완료가 맞는지 자체 확인
 - m_pullCreateMap과 m_pullModify의 모든 pendingState의 completed 값이 true인지
 - false인 pendingState가 있으면,
 - 에러 메시지 출력 (또, 어떤 처리?)
 - 모든 pendingState의 completed 값이 true이면
 - 인메모리 client-index 정보를 파일로 저장 (메모리 값은 op별로 완료되었을 때 업데이트 된 상태)
 - cursor를 serverCursor로 업데이트하고 file로 저장
 - pullCreateMap, pullDeleteMap, pullModifyMap 내용 비우기
 - 동기화 세션 상태 초기화
 - pendingPushMap이 비어있지 않으면,
 - 동기화 세션 상태 push로 설정
 - 서버로 START_PUSH_ENTRY_LIST 이벤트 전송
 - push 동기화 진행 방안은 아래 참조
- pull 동기화 완료 후, pendingPushMap에 대한 push 동기화 진행 방안클라이언트
 - 서버
 - START_PUSH_ENTRY_LIST 수신 처리
 - START_FILE_LIST 처리와 유사하게 처리
 - START_PUSH_ENTRY_LIST_ACK 클라이언트로 전송
 - 클라이언트
 - START_PUSH_ENTRY_LIST_ACK 수신 처리
 - START_FILE_LIST_ACK 처리와 유사하게 처리
 - PUSH_ENTRIES 서버로 전송
 - 서버
 - PUSH_ENTRIES 수신 처리
 - FILE_ENTRIES 처리와 유사하게 처리
 - PUSH_ENTRIES_ACK 클라이언트로 전송
 - 클라이언트
 - PUSH_ENTRIES_ACK 수신 처리

- FILE_ENTRIES_ACK 처리와 유사하게 처리
- 모든 PUSH_ENTRIES를 전송했으면,
 - 마무리로 END_PUSH_ENTRY_LIST 서버로 전송
- 서버
 - END_PUSH_ENTRY_LIST 수신 처리
 - END_FILE_ENTRY_LIST 처리와 유사하게 처리
 - END_PUSH_ENTRY_LIST_ACK 클라이언트로 전송
 - CMFileSyncPushGenerator (신규) 스레드 시작
- 클라이언트
 - END_PUSH_ENTRY_LIST_ACK 수신 처리
 - END_FILE_LIST_ACK 처리와 유사하게 처리
- 서버의 CMFileSyncPushGenerator 동작
 - <https://chatgpt.com/share/6924126b-f76c-800f-965e-83401a5b7c03>
 - CMFileSyncInfo에 op별 push 동기화용 Map 준비
 - CMFileSyncPendingEntry 클래스 정의
 - 기존 CMFileSyncClientEntry 상속
 - CMFileSyncOp m_opHint 추가
 - CMFileSyncOp enum 정의
 - CREATE, DELETE, MODIFY, UNKNOWN
 - CMFileSyncPendingState 클래스 정의
 - final CMFileSyncPendingEntry entry;
 - boolean completed;
 - CMFileSyncPushOpStateTable 정의
 - Map<path, CMFileSyncPendingState> createMap = new HashMap<>();
 - Map<path, CMFileSyncPendingState> modifyMap = ...
 - Map<path, CMFileSyncPendingState> deleteMap = ...
 - Map<path, CMFileSyncPendingState> conflictMap = ...
 - Map<CMFileSyncStateKey, CMFileSyncPushOpStateTable> m_pushStateTable; 추가 (클라이언트별 push op별 동기화 상태 관리)
 - run() 메소드 동작
 - syncInfo.m_pushStateTable에서 이 stateKey로 대상 클라이언트용 CMFileSyncPushOpStateTable인 pushOpStateTable 구하기
 - 없으면 새로 생성하고 syncInfo.m_pushStateTable에 넣기
 - classifyEntriesForPush(serverSyncHome, indexRegistry) 호출하여 op별 Map 채우기
 - 메소드 내용은 아래 별도 기술
 - if(!pushOpStateTable.deleteMap().empty())
 - 삭제 파일에 대해 삭제하고, 메타 정보 갱신하고, 클라이언트로 완료 이벤트 전송
 - processDeleteFiles()에서 처리
 - 이 메소드는 CMFileSyncManager의 completeDeleteFiles() 와 메타 정보 갱신 및 이벤트 전송 과정 동일

- if(!pushOpStateTable.createMap.empty())
 - 신규 파일에 대해 클라이언트로 전송 요청
 - requestTransferOfNewFiles()에서 처리
 - 이 메소드는 기존 CMFileSyncGenerator의 메소드와 구조 동일
- if(!pushOpStateTable.modifyMap.empty())
 - 업데이트 파일에 대해 클라이언트와 파일별 블록 체크섬 통한 rsync delta 동기화 진행
 - requestDeltaSyncOfModifiedFiles() 호출
 - 이 메소드는 기존 CMFileSyncGenerator의 compareBasisAndClientFileList()와 구조 동일
- pushOpStateTable.conflictMap은 지금 아니고 이 push 동기화 완료될 때 complete 이벤트 전에 별도 이벤트에 담아 보낼 예정
- 메소드 종료 전에 push 동기화 완료 여부 체크
 - 기존 설계처럼 pushOpStateTable.createMap과 pushOpStateTable.modifyMap의 모든 path에 대해 pendingState의 completed 값이 true이면 완료
 - CMFileSyncManager에 isCompletePushSync(...), completePushSync(...) 정의
- private void classifyEntriesForPush(Path serverSyncHome, CMFileSyncIndexRegistry indexRegistry) 동작
 - pendingPushEntryList의 각 원소 entry에 대해 다음 수행
 - Path relPath = entry.getPathRelativeToHome();
Path absPath = serverSyncHome.resolve(relPath);
 - long curMtime = entry.getCurMtime(); // 클라이언트 mtime
boolean curExists = (curMtime != -1);
 - CMFileSyncIndexRepository repo = indexRegistry.getOrLoad(userId, deviceId);
CMFileSyncIndexEntry serverEntry = repo.readEntry(relPath);
long baseMtime = (serverEntry == null || serverEntry.tombstone()) ? -1 : serverEntry.mtime(); // entry의 서버 상의 mtime
boolean baseExists = (baseMtime != -1);
 - relPath의 서버상 절대 경로 absPath 구하기
long serverMtime = Files.exists(absPath) ? Files.getLastModifiedTime(absPath).toMillis() / 1000 : -1;
boolean serverExists = (serverMtime != -1);
 - opHint가 UNKNOWN일 때 (pull 동기화 후 클라에서 작성된 entry 리스트)
 - if(!baseExists && curExists)
 - entry로 CMFileSyncPendingState 만들어서 pushOpStateTable.createMap에 추가
 - if(serverExists)
 - 서버에도 파일 있어서 충돌 처리로 serverEntry를 (conflict...) rename하고 CMFileSyncPendingState 만들고 pushOpStateTable.conflictMap에 추가
 - rename된 파일에 대해 인메모리 동기화 메타 정보 업데이트 (index, cursor, changelog)
 - else if(baseExists && curExists && curMtime != baseMtime)
 - if(serverExists && serverMtime == baseMtime) (서버 파일 있는데 base 이후 수정 안함)
 - entry로 CMFileSyncPendingState 만들고 pushOpStateTable.modifyMap에 추가
 - else if(serverExists && serverMtime != baseMtime) (서버 파일 있고 base 이후 수정도 됨)
 - 충돌 처리로 serverEntry를 (conflict...) rename하고 CMFileSyncPendingState 만들고 pushOpStateTable.conflictMap에 추가

- rename된 파일에 대해 인메모리 동기화 메타 정보 업데이트 (index, cursor, changelog)
 - entry로 CMFileSyncPendingState 만들고 pushOpStateTable.createMap에 추가
- else if(!serverExists) (그냥 else ??)
 - entry로 CMFileSyncPendingState 만들고 pushOpStateTable.createMap에 추가
- else
 - 클라이언트 파일 수정 없거나 삭제 상태이므로 무시
- opHint가 CREATE일 때 (WatchService에서 탐지)
 - if(!baseExists && curExists)
 - entry로 CMFileSyncPendingState 만들고 pushOpStateTable.createMap에 추가
 - if(serverExists)
 - 충돌 처리로 serverEntry를 (conflict...) rename하고 CMFileSyncPendingState 만들고 pushOpStateTable.conflictMap에 추가
 - rename된 파일에 대해 인메모리 동기화 메타 정보 업데이트 (index, cursor, changelog)
 - else if(baseExists && curExists)
 - if(serverExists && serverMtime == baseMtime)
 - entry로 CMFileSyncPendingState 만들고 pushOpStateTable.modifyMap에 추가
 - else if(serverExists && serverMtime ≠ baseMtime)
 - 충돌 처리로 serverEntry를 (conflict...) rename하고 CMFileSyncPendingState 만들고 pushOpStateTable.conflictMap에 추가
 - rename된 파일에 대해 인메모리 동기화 메타 정보 업데이트 (index, cursor, changelog)
 - entry로 CMFileSyncPendingState 만들고 pushOpStateTable.createMap에 추가
 - else if(!serverExists) (그냥 else ??)
 - entry로 CMFileSyncPendingState 만들고 pushOpStateTable.createMap에 추가
- opHint가 MODIFY일 때 (WatchService에서 탐지)
 - if(baseExists && curExists && baseMtime ≠ curMtime)
 - if(serverExists && baseMtime == serverMtime)
 - entry로 CMFileSyncPendingState 만들고 pushOpStateTable.modifyMap에 추가
 - else if(serverExists && baseMtime ≠ serverMtime)
 - 충돌 처리로 serverEntry를 (conflict...) rename하고 CMFileSyncPendingState 만들고 pushOpStateTable.conflictMap에 추가
 - rename된 파일에 대해 인메모리 동기화 메타 정보 업데이트 (index, cursor, changelog)
 - entry로 CMFileSyncPendingState 만들고 pushOpStateTable.createMap에 추가
 - else if(!serverExists) (그냥 else ??)
 - entry로 CMFileSyncPendingState 만들고 pushOpStateTable.createMap에 추가
 - else if(!baseExists && curExists)
 - entry로 CMFileSyncPendingState 만들고 pushOpStateTable.createMap에 추가
 - if(serverExists)
 - 충돌 처리로 serverEntry를 (conflict...) rename하고 CMFileSyncPendingState 만들고 pushOpStateTable.conflictMap에 추가
 - rename된 파일에 대해 인메모리 동기화 메타 정보 업데이트 (index, cursor, changelog)
- opHint가 DELETE일 때 (WatchService에서 탐지, curMtime = -1로 설정)

- if(baseExists && !curExists)
 - if(serverExists && serverMtime == baseMtime)
 - entry로 CMFileSyncPendingState 만들고 pushOpStateTable.deleteMap에 추가
 - 삭제 및 메타 정보 업데이트는 아래 별도 메소드에서 일괄 진행
 - else if(serverExists && serverMtime ≠ baseMtime)
 - 충돌 처리로 serverEntry를 (conflict...) rename하고 CMFileSyncPendingState 만들고 pushOpStateTable.conflictMap에 추가
 - rename된 파일에 대해 인메모리 동기화 메타 정보 업데이트 (index, cursor, changelog)
- 서버의 CMFileSyncPushGenerator가 위에서 생성한 pushOpStateTable.(create/modify/delete/conflict)Map에 대한 처리 메소드
 - processDeleteFiles()
 - 여기서 Map 내 key인 path들 일괄 삭제
 - 메타 정보 업데이트 처리 (index 파일에서 tombstone = true, version++, mtime 최신 수정, lastChangeld 증가)
 - 클라이언트에게 COMPLETE_DELETE_FILES 이벤트 전송으로 삭제 완료 알림
 - push 동기화 완료 여부 체크
 - 삭제 과정 빠고 나머지는 9번 설계 문서의 completeDeleteFiles()와 내용 동일
 - requestTransferOfNewFiles()
 - 기존 CMFileSyncGenerator의 requestTransferOfNewFiles()와 동일 기능으로 Map 내 각 key인 path에 대해 클라이언트에게 파일 전송 (push) 요청
 - 파일 별로 전송 완료될 때마다 메타 정보 업데이트 예정
 - requestDeltaSyncOfModifiedFiles()
 - 기존 CMFileSyncGenerator의 compareBasisAndClientFileList()와 동일 기능으로 Map 내 각 key인 path에 대해 delta sync 작업 진행
 - 파일 별로 업데이트 완료될 때마다 메타 정보 업데이트 예정
- 서버가 클라로 신규 파일 전송 요청하고 각 파일의 전송이 완료될 때마다의 동작
 - 서버
 - 로직이 기존 rsync push 기반 (최초) 동기화에서 서버로 신규 파일 전송 완료시의 동작을 기반으로 함
 - CMFileSyncManager.checkNewTransferForSync()에서 다음 작업
 - 파일 전송 완료한 곳에서 최초 동기화인지 양방향 push용 동기화인지 확인
 - 내가 서버이고, CMFileSyncPushGenerator 검색 및 이 클라이언트의 stateKey에 대한 pushOpStateTable.createMap()에서 전송 완료 path에 대한 pendingState 있는지 확인
 - push용 동기화하면 completeNewFileTransferForPush() 추가하여 아래 작업
 - 전송 파일을 sync home으로 이동하고 modified time 복원 (기존 최초 동기화와 동일)
 - 서버 메타 정보 갱신 (인메모리나 파일 업데이트 정책은 9번 문서 내용 따름, index/changelog/cursor)
 - 해당 path의 pendingState의 completed 값을 true로 변경
 - 클라이언트로 COMPLETE_NEW_FILE 전송 (기존 최초 동기화와 동일)
 - push 동기화 세션 전체 완료 여부 검사
 - CMFileSyncManager.isCompletePushFileSync() 이용 (기존 최초 동기화와 로직 유사)
 - 이 클라이언트용 pushOpStateTable내의 createMap, modifyMap 상태가 모두 true인지 여부 확인
 - push 동기화 세션 전체 완료되었으면

- CMFileSyncManager.completePushFileSync() 호출하여 완료 작업 (기존 최초 동기화와 로직 유사)
- 클라이언트
 - COMPLETE_NEW_FILE 이벤트 수신
 - 최초 동기화용인지 양방향 push 동기화용인지 구분
 - 동기화 상태가 push인지, 클라이언트측 pendingPushMap에 해당 path가 있는지 여부 확인
 - 있으면 해당 pendingState의 completed 값을 true로 변경
 - 완료 파일에 대한 index 및 cursor 메타 정보 업데이트 (9번 문서 내용 따름)
- 클라이언트로부터 업데이트 파일의 delta 전송이 완료될 때마다 서버의 동작
 - 서버
 - 로직이 기존 rsync push 기반 (최초) 동기화에서 서버로 파일 delta 업데이트 완료시의 동작을 기반으로 함
 - CMFileSyncEventHandler.processEND_FILE_BLOCK_CHECKSUM_ACK()에서 이 업데이트 완료가 기존 최초 동기화용 완료인지 여기서의 push용 완료인지 구분
 - 서버이고 generator와 업데이트 상태 list 또는 map 존재 여부로 판단
 - 최초 동기화용 업데이트이면, 기존의 CMFileSyncManager.completeUpdateFileForSync() 호출
 - push용 업데이트이면, CMFileSyncManager.completeUpdateFileForPush()에서 다음 작업 진행
 - CMFileSyncPushGenerator 객체, pushOpStateTable 조회
 - 업데이트된 temp file을 sync home 파일로 반영
 - 메타 정보 갱신 (index, cursor, changelog 등 9번 문서 내용에 따름)
 - pushOpStateTable.modifyMap의 pendingState의 completed 값을 true로 변경
 - 클라이언트로 COMPLETE_UPDATE_FILE 이벤트 전송 (9번 문서 참고)
 - push 동기화 세션 전체 완료 여부 검사
 - 위 신규 파일 전송 완료 때와 동일 로직
 - push 동기화 세션 전체 완료되었으면
 - CMFileSyncManager.completePushFileSync() 호출 (동작은 아래 참고)
 - 클라이언트
 - COMPLETE_UPDATE_FILE 이벤트 수신 후 다음과 같이 처리
 - 최초 동기화용인지 push용인지 구분
 - pendingPushMap에 해당 path 있으면 push용, 기존 completeMap에 path 있으면 최초 동기화
 - 최초 동기화용이면 기존 코드 그대로 진행 (메소드로 분리?)
 - push용 동기화이면 다음 진행 (메소드로 분리?)
 - pendingPushMap의 해당 pendingState의 completed 값을 true로 변경
 - client-index와 cursor 메타 정보 업데이트 (9번 문서 내용 따름)
- 이 push 동기화 세션이 완료되었다고 판단할 때 동작 (isCompletePushFileSync())
 - 서버
 - pushOpStateTable.conflictMap에 내용이 있으면 클라이언트로 전송 시작
 - START_PUSH_CONFLICT_LIST 전송
 - 필드: int nNumConflictFiles
 - 없으면 바로 CMFileSyncManager.completePushFileSync() 호출
 - 클라이언트
 - START_PUSH_CONFLICT_LIST 수신

- START_PUSH_CONFLICT_LIST_ACK 전송
 - 필드: int nReturnCode = 1
- 서버
 - START_PUSH_CONFLICT_LIST_ACK 수신
 - PUSH_CONFLICT_ENTRIES 전송 (entry 개수가 많을 때는 나눠서 반복 전송)
 - 필드: List<Path> relativePathList
- 클라이언트
 - PUSH_CONFLICT_ENTRIES 수신
 - PUSH_CONFLICT_ENTRIES_ACK 전송
 - 필드: int nReturnCode = 1, List<Path> relativePathList
- 서버
 - PUSH_CONFLICT_ENTRIES_ACK 수신
 - m_pushStatTable에서 pushOpStateTable을 찾고 거기서 conflictMap 구하기
 - 수신 완료한 path들에 대해서 conflictMap에서 CMFileSyncPendingState.completed 값을 true로 변경
 - conflictMap의 모든 path에 대해 pendingState.completed 값이 true가 되면, END_PUSH_CONFLICT_LIST 전송
 - 필드: int nNumConflictFiles
- 클라이언트
 - END_PUSH_CONFLICT_LIST 수신
 - END_PUSH_CONFLICT_LIST_ACK 전송
 - 필드: int nReturnCode = 1
- 서버
 - END_PUSH_CONFLICT_LIST_ACK 수신
 - CMFileSyncManager.completePushFileSync() 호출
- 이 push 동기화 세션이 완료되었을 때 동작
 - 서버
 - CMFileSyncManager.completePushFileSync()에서 다음 진행
 - push 세션 완료 여부 확인
 - m_pushStateTable에서 해당 클라이언트의 pushOpStateTable 구하기
 - pushOpStateTable의 각 createMap, modifyMap, deleteMap, conflictMap의 모든 pendingState.completed 값이 true인지 확인
 - 메타 정보 안정화 (index 및 changelog를 파일로 flush)
 - 클라이언트에게 COMPLETE_PUSH_SYNC 이벤트 전송
 - 필드: int numFilesCompleted
 - 이 클라이언트용 push 동기화 상태 초기화
 - pushGenerator 삭제, pushOpStateTable 삭제
 - 클라이언트
 - COMPLETE_PUSH_SYNC 수신하여 다음 작업 진행
 - pendingPushMap 검사하여 모두 완료되었는지 확인
 - client-index를 파일로 flush

- pendingPushMap, 상태 변수 등 초기화
 - 동기화 세션 상태 idle로 전환
 - (최초 동기화 처럼) WatchService에서 감지된 파일 변화가 있다면 push 동기화 다시 시작
- 클라이언트가 동기화 디렉토리에서 파일 변경(추가,삭제,수정) 감지한 경우 동기화 과정
 -
 - (**from here**)
- 다중 로그인 사용자의 디바이스간 파일 모드 동기화 메카니즘
 - 클라이언트의 두 번째 이후 디바이스에 대해서는 pull 동기화시에 온라인 모드로 처리해서 동기화 속도 향상
 - (not yet)